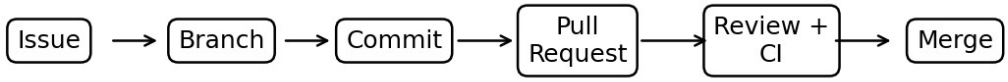
37. GIT & GITHUB

Commit • Branch • Merge • Pull Request • Issues • Collaboration • CI

Profesyonel GitHub iş akışı



Small PR • automated checks • human review • traceable history

Bu modülün amacı
Git'i sadece dosya yedekleme aracı olarak değil, profesyonel yazılım ve veri bilimi geliştirme sürecinin omurgası olarak öğrenmek. Commit geçmişi, branch stratejisi, review, issue takibi, CI ve repository yapısını gerçek ekip pratiği seviyesinde kullanmak.

Kazanım	Modül sonunda
Git mental model	Working tree, staging area, commit ve branch ilişkisini açıklayabilirsin.
Günlük kullanım	add/commit/push/pull/restore/revert/rebase komutlarını güvenle kullanabilirsin.
Collaboration	Branch, Pull Request, review ve conflict çözme akışını yönetebilirsin.
Data science repo	Notebook, source, tests, configs ve data dosyalarını doğru düzenleyebilirsin.
GitHub	Issues, labels, releases, Actions ve repository settings mantığını kullanabilirsin.
Güvenlik	Secrets ve büyük veriyi Git history'ye sokmadan çalışabilirsin.

1. Git ve GitHub nedir?

Git, dosyaların zaman içindeki sürümlerini commit adı verilen snapshot'lar halinde tutan distributed version control sistemidir. GitHub ise Git repository'lerini barındırır ve ekip çalışması, review, issues, CI/CD ve releases sağlar.

Git != GitHub
Git version control teknolojisidir; GitHub ise bu teknolojiyi ekip iş akışıyla çevreleyen platformdur.

2. Git mental modeli

Alan	Anlam
Working tree	Diskte düzenlediğin dosyalar
Staging area	Bir sonraki commit'e girecek seçili değişiklikler
Repository / HEAD	Kaydedilmiş commit geçmişi ve aktif commit

```
git status
git add src/model.py
git commit -m "Add churn baseline model"
```

Commit = snapshot
Commit yalnız değişiklik listesi değil; repository durumunun versioned snapshot'ıdır.

3. Repository oluşturma

```
mkdir churn-project
cd churn-project
git init
```

```
git clone https://github.com/org/churn-project.git
```

init vs clone
git init yeni yerel repo başlatır; git clone mevcut remote repository'nin geçmişini kopyalar.

4. Günlük temel komutlar

Amaç	Komut
Durum	git status
Değişiklik	git diff
Stage	git add <file>
Commit	git commit -m "..."
Geçmiş	git log --oneline --graph

5. Staging neden var?

Aynı çalışma oturumunda yaptığın tüm değişiklikleri değil, mantıksal olarak birlikte olan parçaları tek commit halinde seçebilmeni sağlar.

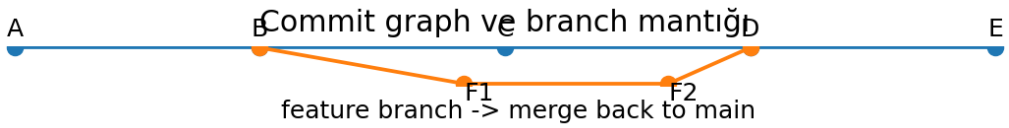
```
git add src/features.py
git add tests/test_features.py
```

Profesyonel alışkanlık
Commitleri küçük, tek amaçlı ve review edilebilir tut.

6. İyi commit mesajı

Kötü	Daha iyi
fix	Fix null handling in customer feature pipeline
model	Add LightGBM churn baseline
update notebook	Document feature-ablation experiment
stuff	Remove leaked post-churn cancellation feature

7. Branch nedir?



Branch, commit graph üzerindeki hareketli bir pointer'dır. Yeni özellik üzerinde main branch'i bozmadan çalışmayı sağlar.

```
git switch -c feature/churn-threshold
```

8. Branch isimleri

Pattern	Örnek
feature/	feature/add-shap-report
fix/	fix/sql-null-filter
docs/	docs/update-readme
refactor/	refactor/training-pipeline

9. Merge

```
git switch main
git merge feature/churn-threshold
```

Amaç
İki branch'in commit geçmişini tek graph içinde birleştirmek.

10. Merge conflict

Aynı dosyanın aynı bölgesinde uyumsuz değişiklik varsa Git otomatik birleştiremez.

```
<<<<<<< HEAD
threshold = 0.30
=======
threshold = 0.25
>>>>>>> feature/tune-threshold
```

1. git status ile conflict dosyalarını gör.
2. Doğru nihai içeriği düzenle.
3. Conflict markerlarını sil.
4. Testleri çalıştır.
5. git add ile resolved işaretle.
6. Merge veya rebase sürecini tamamla.

11. Rebase

Bir branch'in commitlerini yeni bir base commit üzerine yeniden uygular; daha lineer history sağlar.

```
git fetch origin
git rebase origin/main
```

History rewrite
Rebase commit hash'lerini değiştirir. Ortak kullanılan public branch'te dikkatsiz rebase/force-push yapma.

12. Merge vs rebase

Kriter	Merge	Rebase
History	Branch graph korunur	Lineerleşir
Hash değişimi	Hayır	Evet
Public branch	Daha güvenli	Dikkat
Local feature sync	Olur	Çok yaygın

13. fetch, pull ve push

Komut	Ne yapar?

git fetch	Remote refs indirir; working tree değiştirmez
git pull	fetch + merge/rebase
git push	Local commitleri remotea gönderir

```
git fetch origin
git pull --rebase origin main
git push -u origin feature/add-rag
```

Kontrollü senkronizasyon
Önce fetch edip log/diff incelemek kör pull yapmaktan daha kontrollüdür.

14. Remote ve origin

```
git remote -v
git remote add origin git@github.com:org/repo.git
```

origin
Sadece geleneksel remote adıdır; teknik olarak farklı isim kullanılabilir.

15. SSH vs HTTPS

Yöntem	Not
HTTPS	Token/credential manager ile erişim
SSH	Key tabanlı geliştirici erişimi

16. .gitignore

```
.venv/
__pycache__/
.ipynb_checkpoints/
.env
data/raw/
models/*.pkl
*.log
```

Önemli
gitignore yalnız henüz tracked olmayan dosyaları engeller. Daha önce commit edilmiş secret veya data historyden otomatik silinmez.

17. Data science için ne commit edilir?

Commit et	Commit etme
src code	Raw private dataset
README	API keys / .env

Tests	Huge model checkpoints
Small sample data	Cache/temp files
Configs without secrets	Large notebook outputs

Kural
Code ve metadata Gitte; büyük/özel data ve artifact external storageda.

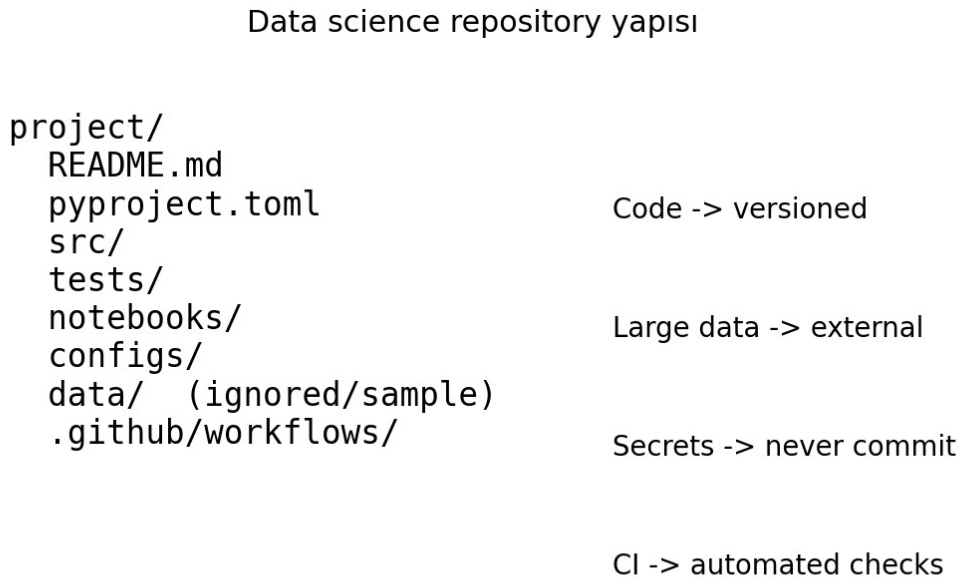
18. Git LFS ve DVC

Git LFS büyük binary dosyalar için pointer tabanlı extensiondır. DVC ise dataset/model artifact version bilgisini Git ile eşleyip içeriği remote storageda tutmaya yardımcı olur.

```
git lfs track "*.onnx"
git add .gitattributes
```

ML reproducibility
Commit SHA + data version + config birlikte experimenti tekrar üretilebilir hale getirir.

19. Data science repository yapısı



```
project/
  README.md
  pyproject.toml
  src/
  tests/
  notebooks/
  configs/
  data/
  .github/workflows/
  .gitignore
```

Notebook tek başına proje değildir
Reusable logic src altında; notebook exploration ve communication için kullanılmalı.

20. README

- Problem ve business amacı.
- Dataset açıklaması.
- Kurulum.
- Reproduce komutları.
- Sonuçlar ve metrics.
- Project structure.
- Limitations.

Portföy
README proje kodundan önce görülen ana ürün sayfasıdır.

21. Environment ve dependency

```
python -m venv .venv
pip install -r requirements.txt
```

Hedef
Environment yeniden üretilebilir olmalı. requirements.txt veya pyproject.toml gibi manifestler version control altında tutulmalı.

22. Config dosyaları

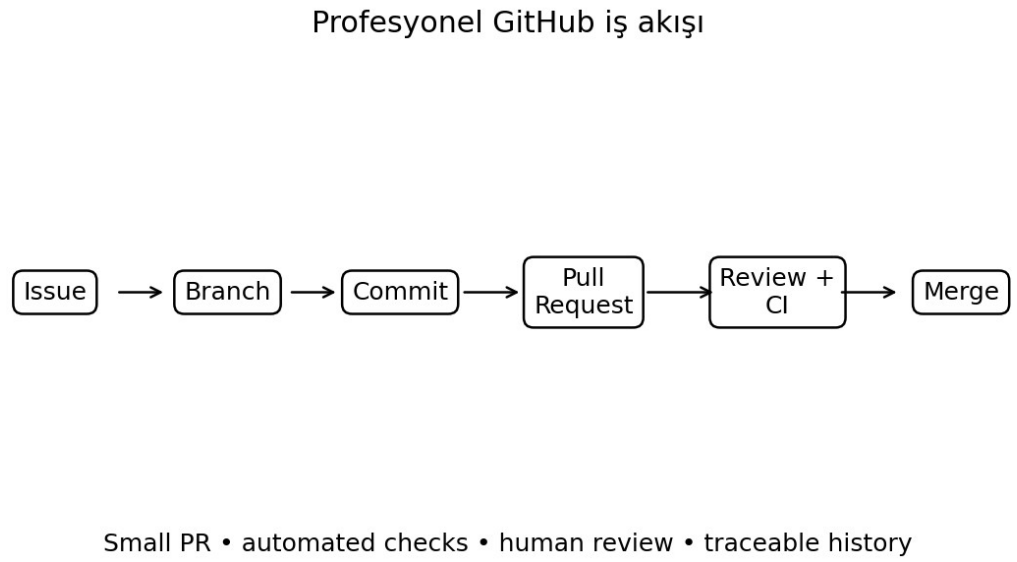
```
seed: 42
model:
  type: lightgbm
  learning_rate: 0.05
  num_leaves: 31
```

Config != secret
Hyperparameter config repoda; password ve API token environment/secrets managerda.

23. GitHub Issues

İyi issue	İçerik
Başlık	Add time-based cross validation
Context	Neden gerekiyor?
Acceptance criteria	Hangi sonuç tamam sayılacak?
Labels	bug/feature/data/docs
Owner	Kim yapıyor?

24. Pull Request



PR ne sağlar?
Change set için tartışma, code review, CI sonucu ve audit trail.

25. İyi PR

- Küçük ve tek amaçlı.
- Açık başlık/açıklama.
- Test sonucu var.
- Breaking change belirtilmiş.
- Metric/screenshot gerekiyorsa eklenmiş.
- İlgili issue linklenmiş.

Draft PR
Erken feedback gerekiyorsa draft PR açmak profesyonel bir alışkanlıktır.

26. Code review - ML özel

Review sorusu	Örnek
Correctness	Leakage var mı?
Readability	Naming net mi?
Tests	Edge case test edilmiş mi?
Performance	Gereksiz full scan var mı?
Security	Secret loglanıyor mu?
Reproducibility	Seed/config/data version kayıtlı mı?

27. Labels ve project board

- bug
- feature
- documentation
- data
- ml
- priority-high

Kanban
Issue ve PR durumunu To do / In progress / Review / Done olarak görünür yapabilirsin.

28. Tags ve releases

```
git tag -a v1.0.0 -m "First production model"
git push origin v1.0.0
```

Tag
Belirli source code stateini sabitler. Release notes ve artifact ile yayınlanabilir.

29. revert, reset, restore

Komut	Kullanım
git revert	Public historyde yeni ters commit oluşturur
git reset	Branch pointerını taşır; local history rewrite
git restore	Working tree/staging dosyasını geri alır

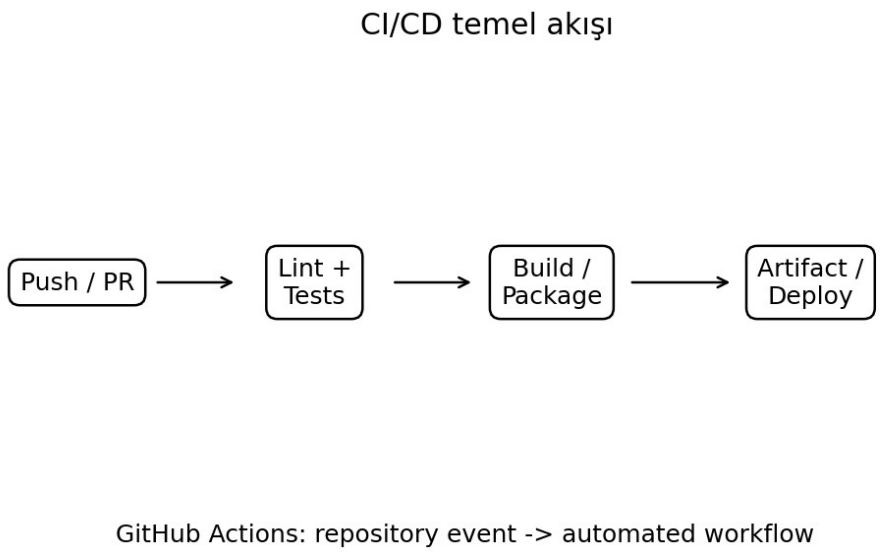
Public main
Yanlış production commitini geri almak için revert genellikle reset --harddan daha güvenlidir.

30. stash, cherry-pick, bisect

```
git stash push -m "wip experiment"
git cherry-pick a1b2c3d
git bisect start
```

Komut	Amaç
stash	Geçici değişiklikleri rafa kaldır
cherry-pick	Tek commit al
bisect	Bug başlangıcını binary search ile bul

31. GitHub Actions




```
name: tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest -q
```

32. CI kalite kapıları

- pytest

- ruff / lint
- format check
- type check
- secret scan
- dependency audit

Branch protection
Main branch merge için CI pass ve review zorunluluğu konabilir.

33. Pre-commit hooks

```
pre-commit install
pre-commit run --all-files
```

Yerel feedback
Pre-commit hızlı feedback sağlar; CI merkezi source of truth olmaya devam eder.

34. Secret scanning

- API key
- Cloud access key
- Database password
- Private token

Secret commit edilirse
Sadece dosyadan silmek yetmez. Credential rotate edilmeli ve gerekiyorsa Git history temizlenmelidir.

35. Branch protection ve CODEOWNERS

```
# .github/CODEOWNERS
src/features/ @data-team
infra/ @ml-platform-team
```

Review ownership
Belirli pathlerde doğru ekip otomatik reviewer olarak atanabilir.

36. Fork workflow ve open source

Write erişimi olmayan contributor kendi forkunda branch açar ve upstream repositoryye Pull Request gönderir.

37. GitFlow, trunk-based ve GitHub Flow

Yaklaşım	Özellik
GitFlow	develop/release/hotfix branchleri; daha ağır
Trunk-based	Kısa ömürlü branch + sık main entegrasyonu
GitHub Flow	main + feature branch + PR

Modern takım
Küçük, kısa ömürlü branch + PR çoğu data/ML takımı için basit ve etkilidir.

38. Notebook diff problemi

Jupyter notebook JSON formatı outputs ve metadata nedeniyle Git diffini gürültülü hale getirebilir.

- Notebook outputlarını temizle.
- Reusable codeu src altına taşı.
- Jupytext/nbsstripout gibi araçları değerlendirebilirsin.

Reproducibility
Notebook hidden state içeriyorsa sonuçlar top-to-bottom yeniden çalışmayabilir.

39. Notebook reproducibility checklist

1. Kernel restart.
2. Run all top-to-bottom.
3. Environment sabitle.
4. Random seed sabitle.
5. Data path/config dışsallaştır.

40. ML experiment ve Git

Artifact	Version bilgisi
Code	Git commit SHA
Data	Snapshot/DVC/version
Config	YAML/TOML in Git
Model	Registry/artifact hash
Metrics	Experiment tracker

Reproduce formülü
commit SHA + data version + config + environment = güçlü reproducibility temeli.

41. Commit SHA model metadata

```
import subprocess

sha = subprocess.check_output(
    ["git", "rev-parse", "HEAD"]
).decode().strip()
```

Traceability
Bir modelin hangi kod sürümüyle eğitildiği sonradan kanıtlanabilir.

42. SQL ve Git

```
sql/
  staging/
  marts/
  tests/
```

Production SQL
Transformation SQL dosyaları da code gibi versioned, reviewed ve tested olmalıdır.

43. GitHub repository güvenliği

- 2FA.
- Least privilege collaborators.
- Protected branches.
- Secret scanning.
- Dependency alerts.
- Private repo gerektiğinde private.

Data privacy
Private dataset repositoryye yanlışlıkla commit edilirse repo private olsa bile governance olayıdır.

44. Dependabot ve dependency güvenliği

Dependency manifestlerindeki bilinen güvenlik açıkları için update alertleri veya Pull Requestler üretilebilir.

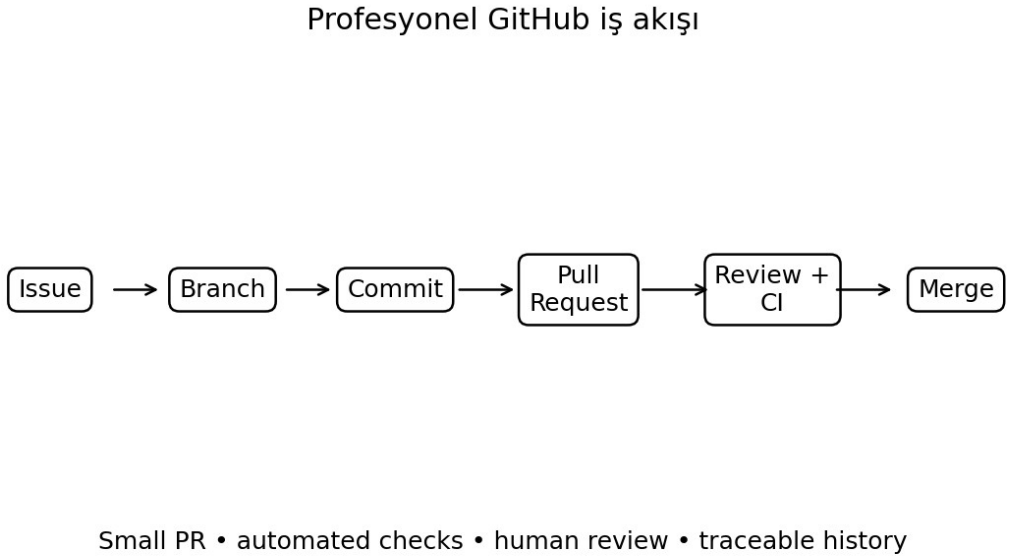
ML supply chain
Python package ve container dependency güvenliği model güvenliğinin parçasıdır.

45. GitHub Pages ve portfolio demo

Statik proje dokümantasyonu veya demo landing page yayınlamak için kullanılabilir.

Portföy
Repo + iyi README + demo linki, yalnız proje adı yazmaktan daha güçlü kanıt üretir.

46. Gerçek takım workflow



1. Issue oluştur.
2. Mainden branch aç.
3. Küçük commitler yap.
4. Push et.
5. Draft/normal PR aç.
6. CI ve review düzeltmelerini yap.
7. Merge et.
8. Branchi sil.
9. Release gerekiyorsa tag oluştur.

47. Data science PR checklist

- Train/validation split doğru mu?
- Leakage var mı?
- Metric business amacına uygun mu?
- Seed/config kaydedildi mi?

- Dataset path hard-coded mı?
- Testler var mı?
- README/experiment result güncel mi?

48. Sık yapılan hatalar

- Main üzerinde direkt çalışmak.
- Her şeyi tek dev commite koymak.
- Dataset/model binarylerini repoya doldurmak.
- Secret commit etmek.
- Conflict çözümünü anlamadan marker silmek.
- Public branchte force push yapmak.
- Notebook outputlarını gereksiz büyütmek.
- README ve reproduce adımlarını yazmamak.
- CI olmadan merge etmek.
- Model artifact ile Git SHAyı ilişkilendirmemek.

49. Günlük komut cheat sheet

Amaç	Komut
Durum	git status
Değişiklik	git diff
Stage	git add <file>
Commit	git commit -m "..."
Branch aç	git switch -c feature/x
Branch değiş	git switch main
Remote güncelle	git fetch origin
Sync	git pull --rebase
Push	git push -u origin feature/x
Geçmiş	git log --oneline --graph
Geri al public	git revert <sha>
Geçici sakla	git stash

50. Mülakat soruları

Soru	Kısa cevap
Git vs GitHub?	Git version control; GitHub hosting/collaboration.
Staging area?	Bir sonraki commite girecek değişiklik seti.
Branch?	Commit graph üzerindeki hareketli pointer.
Merge vs rebase?	Graph birleştirme vs commitleri yeni base üzerine yeniden uygulama.
fetch vs pull?	Fetch remote refs; pull fetch + integration.
revert vs reset?	Yeni ters commit vs history rewrite.
Conflict neden?	Uyumsuz aynı bölge değişiklikleri.
.gitignore?	Untracked dosyaları tracking dışında tutar.
Git LFS?	Large binary pointer sistemi.
PR neden?	Review + CI + traceability.
CI nedir?	Automated build/test checks.
Branch protection?	Main için merge kuralları.
Commit SHA MLde neden?	Modeli kaynak kod sürümüne bağlar.

Secret commit edilirse?	Credential rotate + history cleanup.
Notebook neden zor?	JSON/outputs diff ve hidden-state sorunu.

51. Uygulama alıştırmaları

1. Yeni repo oluştur, README ve .gitignore ekle.
2. Feature branch açıp 3 anlamlı commit yap.
3. İki branch arasında merge conflict üret ve çöz.
4. Feature branchi main üzerine rebase et.
5. Yanlış commiti git revert ile geri al.
6. Issue oluştur ve branch/PR ile kapat.
7. pytest çalışan GitHub Actions workflow yaz.
8. Pre-commit lint/format hook ekle.
9. Notebook outputlarını temizleyip src modülüne refactor et.
10. Training scriptinde Git commit SHAyı experiment metadataya yaz.

52. Mini portföy projesi

Proje: Professional Data Science Repository & Collaboration Workflow

Çıktı	İçerik
README.md	Problem, setup, reproduce, results
src/	Reusable pipeline code
tests/	Unit tests
notebooks/	EDA/demo notebooks
configs/	Experiment configs
.github/workflows/	CI tests + lint
.gitignore	Data/secrets/artifacts
CONTRIBUTING.md	Branch/PR conventions
release v1.0.0	Tagged reproducible state

CV cümlesi
Structured a production-style data science repository with branch-based development, pull-request review, automated CI tests, reproducible configuration, release tagging and Git-linked experiment traceability.

53. Final cheat sheet

Kavram	Hatırlanacak fikir
Working tree	Current files
Stage	Next commit selection
Commit	Immutable snapshot
Branch	Movable commit pointer
Merge	Combine histories
Rebase	Replay on new base
Remote	External repository ref
Fetch	Download refs
Pull	Fetch + integrate
Push	Upload commits
PR	Reviewable change proposal
Issue	Track work/bug
Tag	Named fixed commit
Release	Published version
CI	Automated quality gate
.gitignore	Do not track selected files
Git LFS	Large binary pointers
Branch protection	Protect main
SHA	Exact code version

Bu modülden sonra

38. Software Engineering bölümünde temiz kod, proje yapısı, logging, config, tests ve type hints ile Git repository'lerini production kalitesine taşıyacaksın.